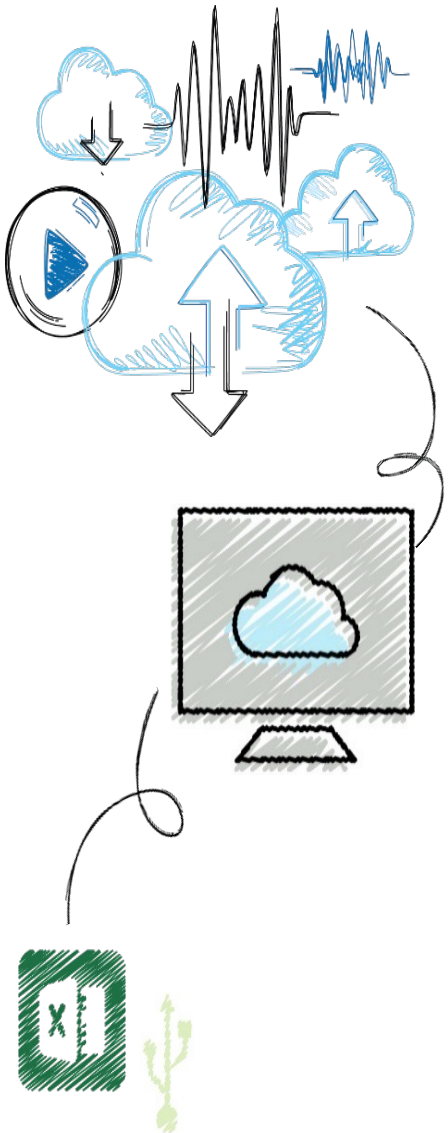
ICT703 Programming

Module 3 Topic 1

Iteration

CENGAGE



Objectives

1. Introduce Iteration
2. Write a loop to repeat a sequence of actions a fixed number of times
3. Write a loop to traverse the sequence of characters in a string
4. Write a loop that counts down and a loop that counts up
5. Write an entry-controlled loop that halts when a condition becomes false

Programming Concepts

Four control flow

- Sequence
- Iteration
- Selection
- Abstraction – methods/classes

- Plus Data

Iteration

- Doing things many times
  - Bounded/Definite
    - Execute commands for a set number of times
    - Python - for loop
  - Unbounded/Indefinite
    - Execute until a condition becomes false
    - Python while loop

Definitive Iteration: The for Loop

- Repetition statements (or **loops**) repeat an action
- Each repetition of action is known as **pass** or **iteration**
- Two types of loops
  - Those that repeat action a predefined number of times
  - Those that perform action until program determines it needs to stop

Executing a Statement a Given Number of Times (1 of 2)

- Python's **for** loop is the control statement that most easily supports definite iteration

```
for eachPass in range(3):
    print("It's alive!")
```

```
It's alive!
It's alive!
It's alive!
```

```
for n in range(2):
    print("It's alive!")
    print("At the moment")
```

```
It's alive!
At the moment
It's alive!
At the moment
```

- The form of this type of loop is:

```
For <variable> in range(<an integer expression>):
    <statement-1>
    .
    <statement-n>
```

- First line of code is sometimes called **loop header**
- Rest of code is **loop body** (must be indented and aligned in the same column)

What do you think this will return?

```
for eachPass in range(4):
    print("It's alive!", str(eachPass))
```

```
It's alive! 0
It's alive! 1
It's alive! 2
It's alive! 3
```

- So you will notice that the range starts at 0 and finishes at 3
- You need to be aware of this when creating your loops as it can lead to incorrect logic if forgotten

Executing a Statement a Given Number of Times (2 of 2)

- Example: Loop to compute an exponentiation for a non-negative exponent

```
number = 2
exponent = 3
product = 1
for eachPass in range(exponent)
        product = product * number
        print(product, end = " ")
2 4 8
```

- If the exponent were 0, the loop body would not execute and value of **product** would remain as 1

Count-Controlled Loops (1 of 2)

- Loops that count through a range of numbers

```
product = 1
for count in range (4):
        product = product * (count + 1)
print(product)

24
```

- To specify a explicit lower bound:

```
product = 1
for count in range (1, 5):
        product = product * count
print(product)

24
```

Count-Controlled Loops (2 of 2)

- Example: bound-delimited **summation**

```
lower = int(input("Enter the lower bound: "))
upper = int(input("Enter the upper bound: "))
theSum = 0
for number in range(lower, upper + 1):
        theSum = theSum + number
print(theSum)
```

```
Enter the lower bound: 1
Enter the upper bound: 10
55
```

What will the result be if we didn't have the +1?

Loop Errors: Off-by-One Error

- Example:

```
# Count from 1 through 4, we think
for count in range(1,4):
    print(count)
```

```
1
2
3
```

- Loop actually counts from 1 through 3
- Not a syntax error, but rather a logic error

Traversing the Contents of a Data Sequence

We can traverse a list of numbers, things, characters and do "stuff"

for <variable> in <sequence>:
<do something with variable>

```
list1 = [1, 2, 3, 4]
for list_element in list1:
        print(list_element)

for fruit in ["apple", "orange", "potato"]:
        print(fruit)

for character in "Hi there!":
      print(character, end = " ")
H i  t h e r e !
```

Specifying the Steps in the Range

- **range** expects a third argument that allows you specify a **step value**

```
theSum = 0
for count in range(2, 11, 2):
        theSum = theSum + count

print(theSum)
30
```

Loops that Count Down

- **The** third argument can be a negative **step value**

```
for count in range(10, 0, –1):
        print(count, end = " ")
10 9 8 7 6 5 4 3 2 1
```

Conditional Iteration: The while Loop

- The **while** loop can be used to describe conditional iteration
  - Example: A program's input loop that accepts values until user enters a **sentinel** that terminates the input

The Structure and Behavior of a While Loop (1 of 3)

- Conditional iteration requires that condition be tested within loop to determine if it should continue
  - Called **continuation condition**
- Syntax:

```
while <condition>:
        <sequence of statements>
```

- Improper use may lead to **infinite loop**
- **while** loop is also called **entry-control loop**
  - Condition is tested at top of loop
  - Statements within loop can execute zero or more times

The Structure and Behavior of a While Loop (2 of 3)

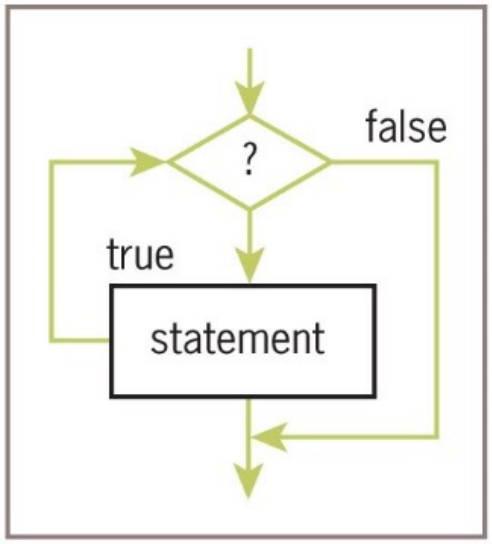


Figure 3-6 The semantics of a **`while`** loop

The Structure and Behavior of a While Loop (3 of 3)

- Example:

```
theSum = 0.0
data = input("Enter a number or just enter to quit: ")
while data != "":
        number = float(data)
        theSum = theSum  + number
        data = input("Enter a number or just enter to quit: ")
print("The sum is", theSum)

Enter a number or just enter to quit: 3
Enter a number or just enter to quit: 4
Enter a number or just enter to quit: 5
Enter a number or just enter to quit:
The sum is 12.0
```

- Data is the **loop control variable**
  - first input statement initializes a variable to a value that the loop condition can test

Count Control with a While Loop

- Use a **while** loop for a count-controlled loop

```
# Summation with a for loop
theSum = 0
for count in range(1, 100001):
        theSum += count
print(theSum)

# Summation with a while loop
theSum = 0
count = 1
while count <= 100000:
        theSum += count
        count += 1
print(theSum)
```

Loop Logic, Errors, and Testing

- Errors to rule out during testing **while** loop:
  - Incorrectly initialized loop control variable
  - Failure to update this variable correctly within loop
  - Failure to test it correctly in continuation condition
- To halt loop that appears to hang during testing, type **Control+c** in terminal window or IDLE shell
- If loop must run at least once, use a **while True** loop with delayed examination of termination condition